

Tracing for Grown-Ups: OpenTelemetry Best Practices for Large Systems

Booking.com

Who am I?



Site reliability engineer at
booking.com



Passionate about technologies



10+ years of backend/DevOps/SRE
experience



Breaking things for good



<https://www.linkedin.com/in/makeitgit/>



Agenda

Foundations of Distributed Tracing

Trace, span, parent/child relationship

Context propagation

Trace context, baggage

Sampling

Head/tail, dynamic sampling



Agenda

Domain modeling

Semantic conventions, span boundaries, naming

Instrumentation strategies

manual/auto, client/server, producer/consumer, log correlation

Advanced instrumentation strategies

Span links, correlation ids, long operations



Agenda

Preprocessing

Enrichment of telemetry data

Instrumentation resiliency

Graceful shutdown

Trace quality

Good trace shape and common failures



Not included in this talk



OpenTelemetry logs



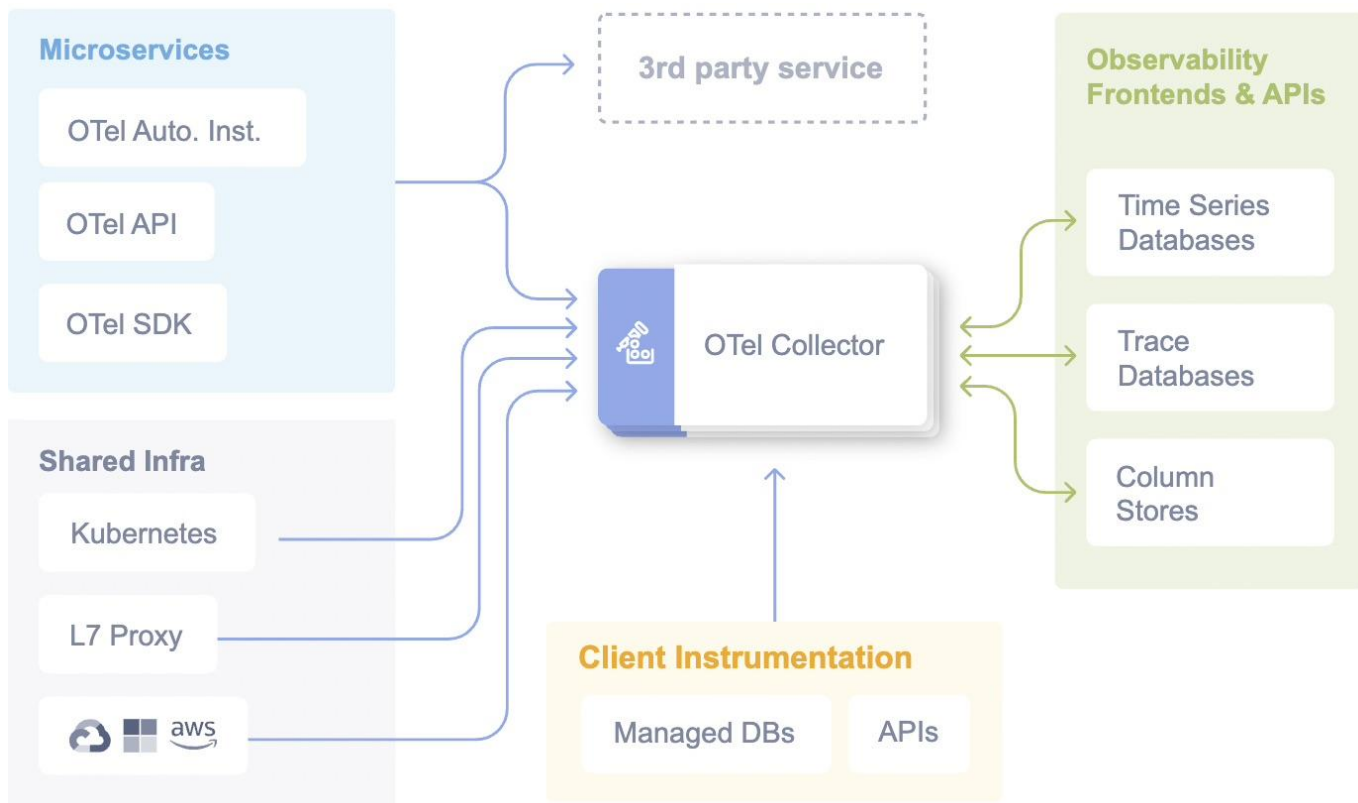
OpenTelemetry metrics



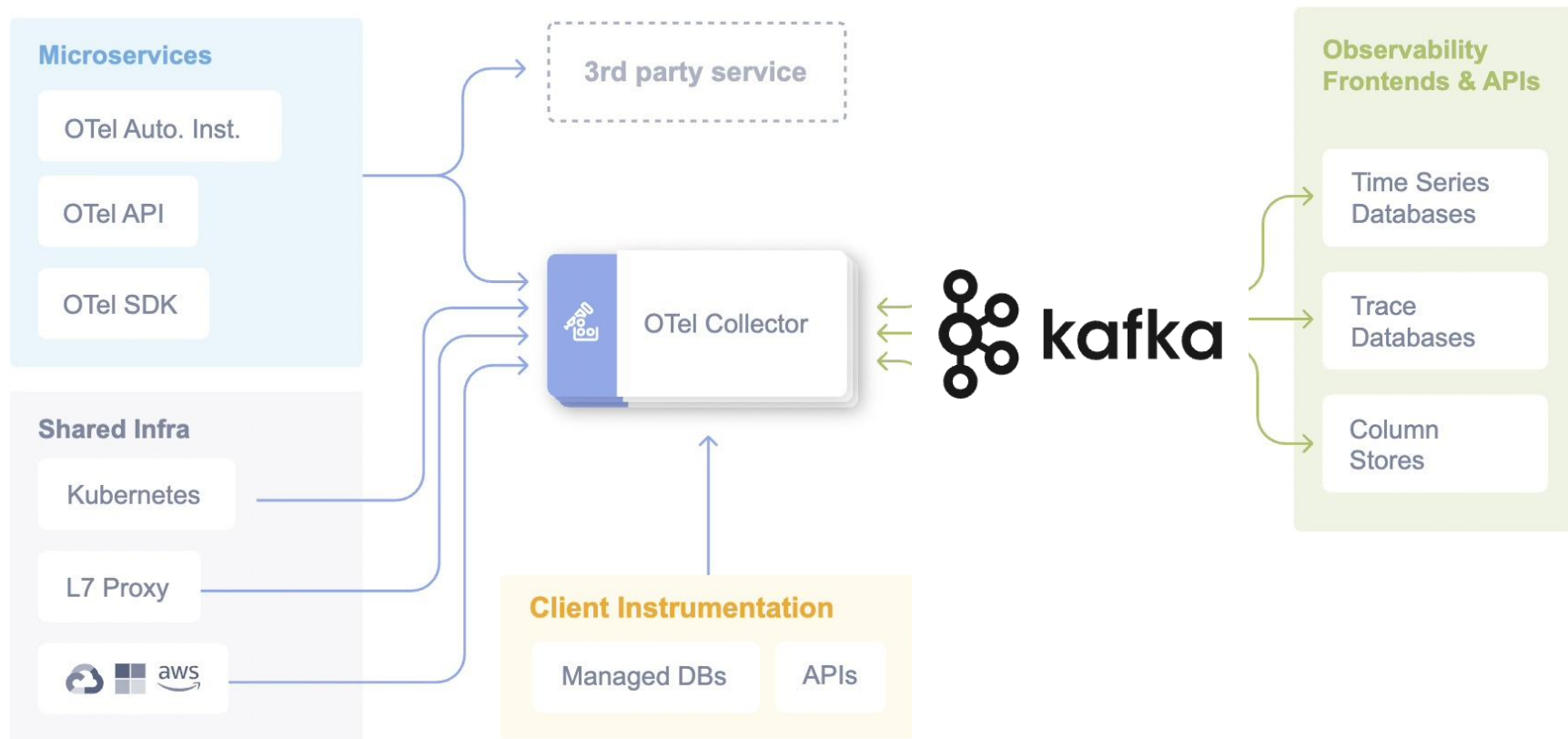
OpenTelemetry profiles

Foundations of Distributed Tracing

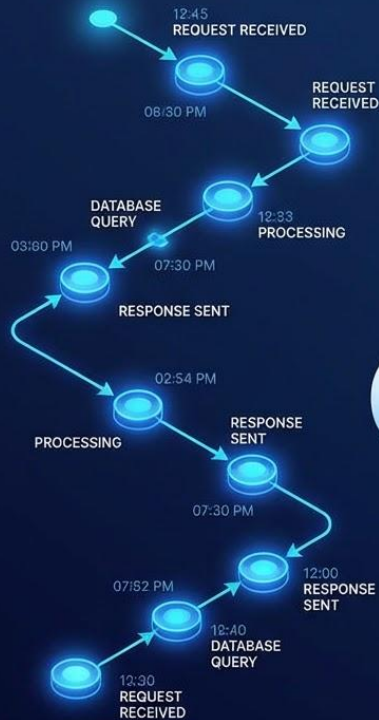
OpenTelemetry architecture



OpenTelemetry architecture at scale

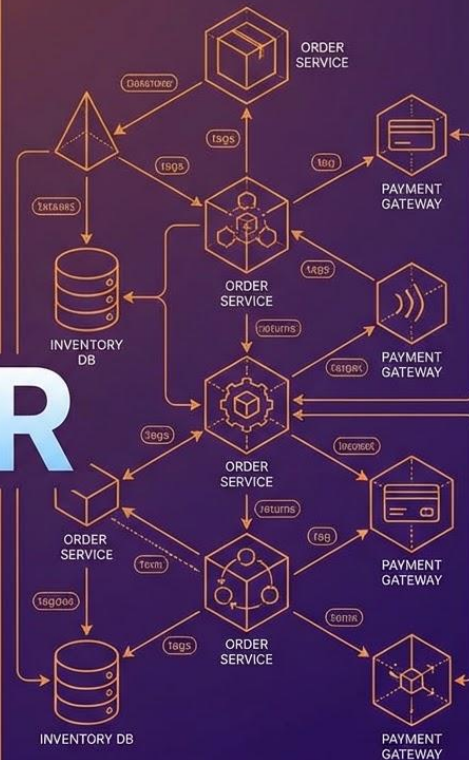


SEQUENCE OF EVENTS



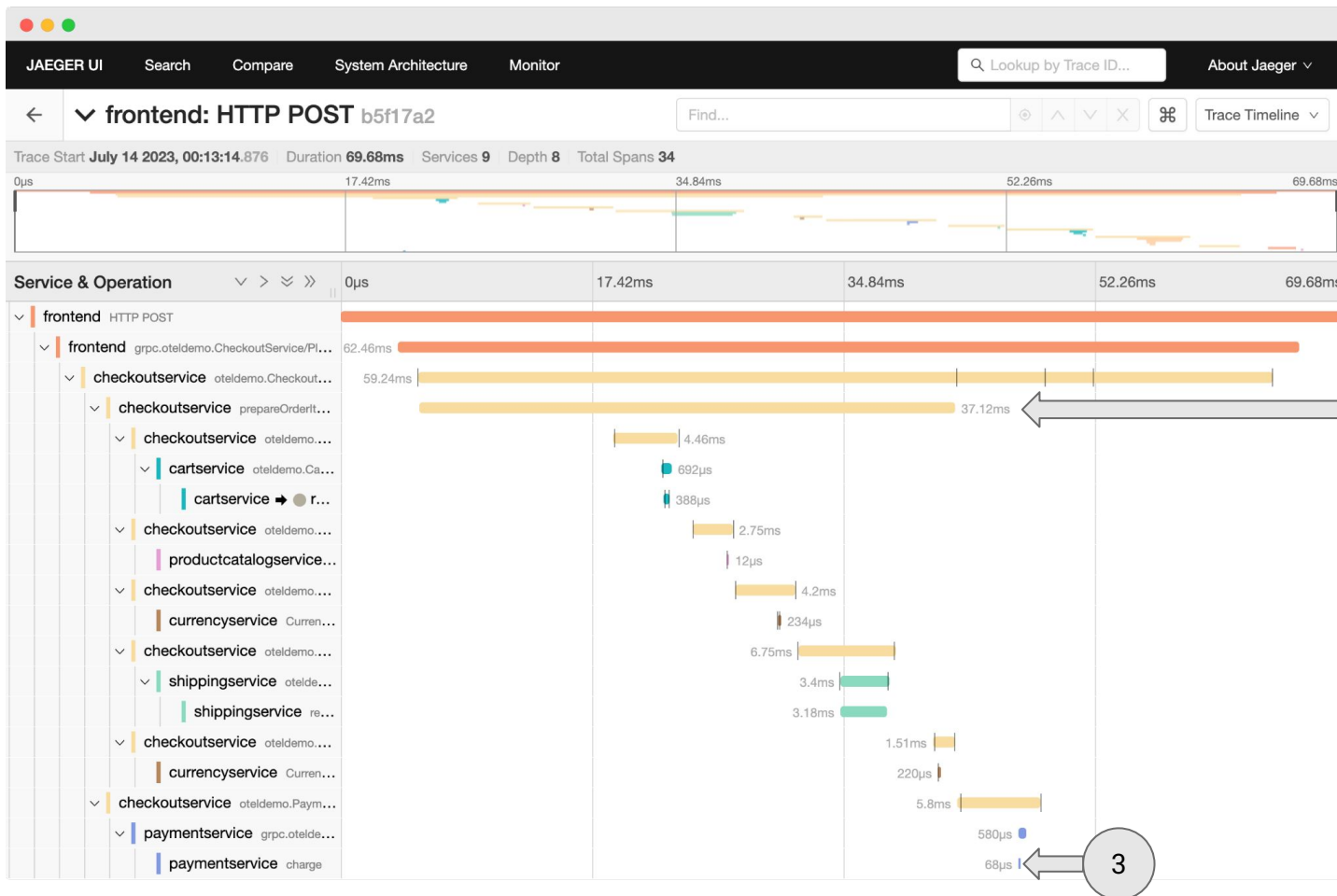
OR

ENCODED DOMAIN MODEL



What is a trace?

What is a trace?



Sequence of operations (spans)

Anatomy of span

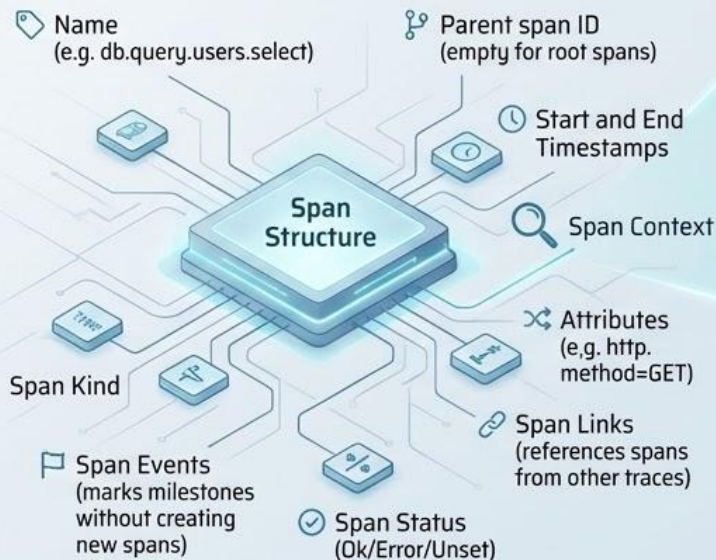
Spans

A single **trace** in distributed tracing consists of a series of tagged time intervals known as **spans**.

Spans represent a logical unit of work in completing some operation.



Structure



Span Context

- Trace ID
- Span ID
- Trace Flags
- Trace State

Span example

```
{  
  name: "GET /api/v1/action",  
  context: {  
    trace_id: "7bba9f33312b3dbb8b2c2c62bb7abe2d",  
    span_id: "086e83747d0e381e",  
  },  
  parent_id: "",  
  start_time: "2021-10-22 16:04:01.209458162 +0000 UTC",  
  end_time: "2021-10-22 16:04:01.209514132 +0000 UTC",  
  status_code: "STATUS_CODE_OK",  
  status_message: "",  
  attributes: {  
    "http.method": "GET",  
    "http.target": "/api/v1/action",  
  },  
  events: [  
    {  
      name: "",  
      message: "OK",  
      timestamp: "2021-10-22 16:04:01.209512872 +0000 UTC",  
    },  
  ],  
}
```

Span kind

Kind	When To Use	Common Attributes
SERVER	Handling an incoming request that originated outside this process (HTTP server, gRPC server)	http.request.method , http.route
CLIENT	Making an outbound call to another service.	url.full , db.query.text
INTERNAL	Pure in-process work.	Domain-specific attributes
PRODUCER	Publishing/sending a message to a broker/queue/stream	messaging.operation.name=send , messaging.destination.name=my-queue
CONSUMER	Receiving/processing a message from a broker/queue/stream	messaging.operation.name=ack

Span kind: why it matters?

LATENCY & METRICS INSIGHTS



Latency breakdown dashboards often separate **SERVER** vs **CLIENT** time.



Many “traces → metrics” jobs derive request rate/error/latency from **SERVER** spans and dependency **SLIs** (e.g. DB, cache) from **CLIENT** spans.



Tail samplers may keep more **SERVER** error/slow spans and aggressively downsample **INTERNAL** noise.

ASYNCHRONOUS & QUEUE VISIBILITY



PRODUCER/CONSUMER clarify asynchronous hops so you can see queue handoff latency.

SYSTEM BOUNDARY CLARITY



Correct kinds make it obvious where a request entered the system and which hops are real network boundaries vs in-process details.

Instrumentation example

```
from flask import Flask, jsonify
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

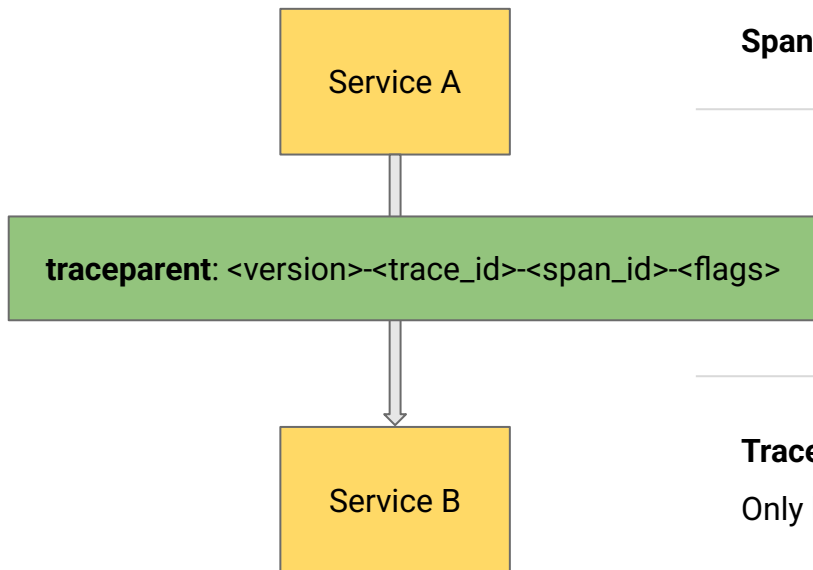
app = Flask(__name__)
tracer = trace.get_tracer(__name__)

@app.route('/api/v1/action')
def risky_operation():
    current_span = trace.get_current_span()

    try:
        result = do_action()
        return jsonify({"result": result})
    except ValueError as e:
        current_span.set_attribute("error.type", "ValueError")
        current_span.set_attribute("error.message", str(e))
        current_span.record_exception(e)
        return jsonify({"error": str(e)}), 400
    except Exception as e:
        current_span.record_exception(e)
        current_span.set_status(Status(StatusCode.ERROR, str(e)))
        return jsonify({"error": "Internal server error"}), 500
```


Context propagation

Trace context

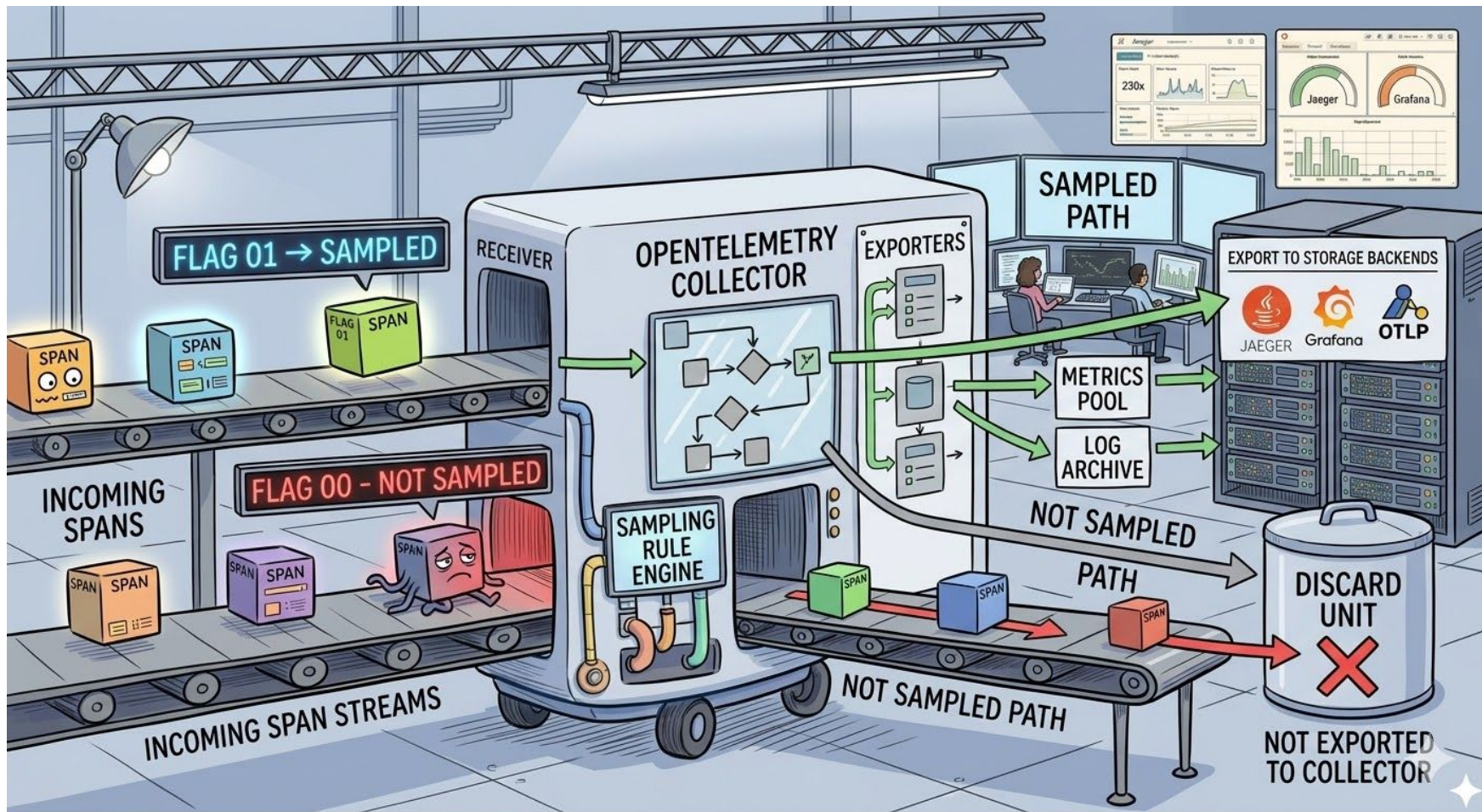


Trace ID - a 128-bit identifier (32 hex chars) shared by all spans in the same trace.

Span ID - a 64-bit identifier (16 hex chars) unique per span within a trace.

Trace Flags - an 8-bit field (2 hex chars) representing trace options.
Only bit 0 is defined: sampled flag.

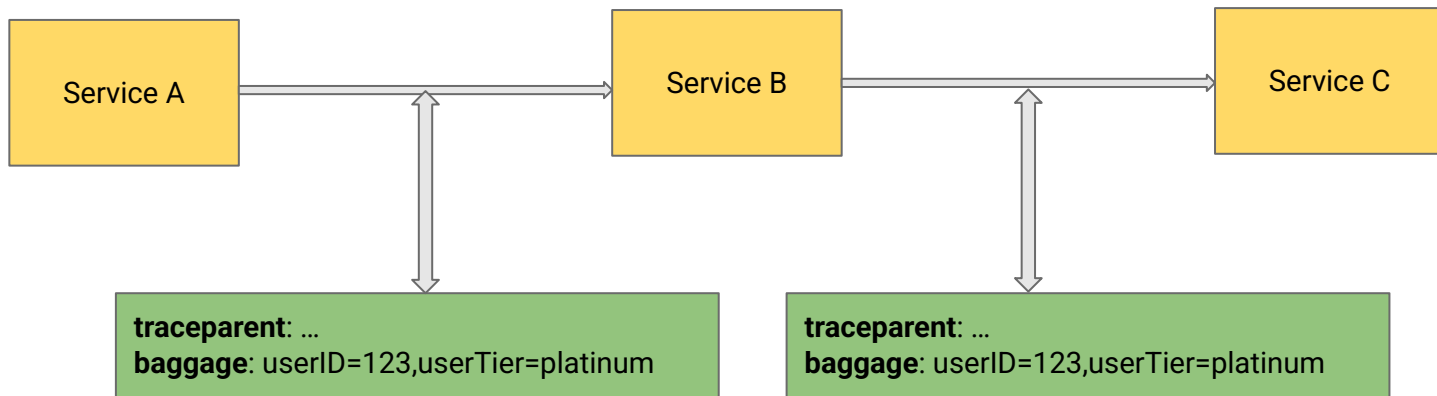
traceparent: <version>-<trace_id>-<span_id>-<flags>

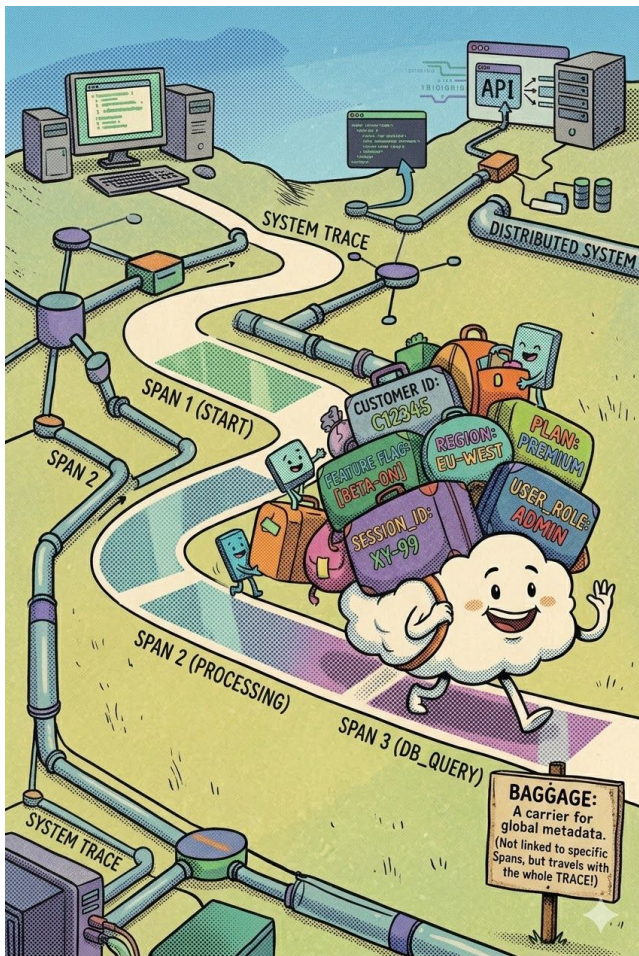


Baggage

OTel baggage is a signal that allows you to pass “global” metadata, like a customer ID or a feature flag.

Baggage is not linked to spans, but rather to the whole trace.





Baggage

Why?

The data being sent from one service is required by a downstream service with multiple services in-between.

How?

Each service must implement logic to parse the data from incoming requests and re-attach it during API calls to downstream services.

Caveat

Use baggage carefully. Avoid passing baggage to external systems to avoid possible leaking of internal data.

Sampling

Sampling

Why?

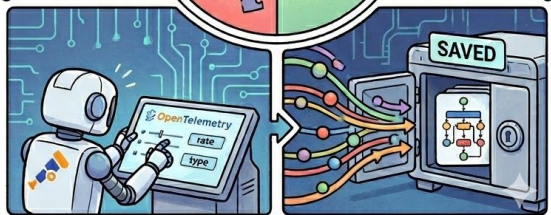
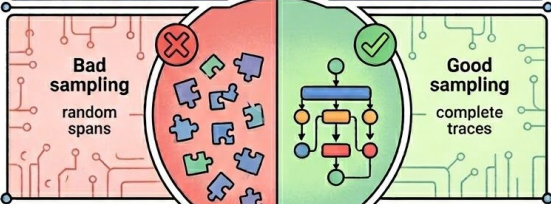
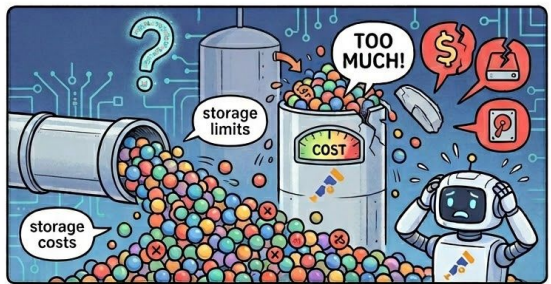
Traces can be very high volume; sampling keeps storage and cost under control.

Goal

Keep traces that are most useful (errors, slow requests, key journeys) while dropping noise.

How?

Good sampling preserves whole traces, not random isolated spans.



Head sampling

Decision is made when the first span is created (e.g., at API gateway).

Common examples:

traceidratio: keep X% of new traces at random.

parentbased_*: if parent says “sampled”, keep; else apply local rule.

Pros:

Reduces overhead early, less data sent downstream.

Cons:

Decision is made before you know if the request will error or be slow.

You may drop many of the most interesting traces (errors, high latency).

Head sampling

```
from opentelemetry.sdk.trace.sampling import (  
    ParentBased,  
    TraceIdRatioBased,  
    ALWAYS_ON,  
    ALWAYS_OFF,  
)  
  
ratio = float(os.getenv("OTEL_TRACES_SAMPLER_ARG", "1.0")) 1  
  
sampler = ParentBased(TraceIdRatioBased(ratio)) 2  
  
resource = Resource.create({"service.name": service_name}) 3  
provider = TracerProvider(resource=resource, sampler=build_sampler())
```

“Smart” head sampling

```
class PrioritySampler(Sampler):
    def _ratio_for_level(self, level: str | None) -> float:
        level = (level or "").lower()
        if level == "high":
            return 1.0
        if level == "medium":
            return 0.5
        return 0.1

    def _extract_priority(self, attributes: dict | None) -> str | None:
        target = attributes.get("http.target")
        parsed = urlparse(target)
        params = parse_qs(parsed.query)
        return params.get("priority")[0]

    def should_sample(self, parent_context, trace_id, name, kind,
attributes, links):
        priority = self._extract_priority(attributes)
        ratio = self._ratio_for_level(priority)

        # Delegate to TraceIdRatioBased for deterministic sampling
        ratio_sampler = TraceIdRatioBased(ratio)
        return ratio_sampler.should_sample(
            parent_context, trace_id, name, kind, attributes, links
        )
```

```
def build_sampler() -> trace.Span:
    """
    - If there is a parent → follow parent
    decision (ParentBased).
    - If there is no parent → decide based
    on HTTP query param `priority`.
    """
    return ParentBased(PrioritySampler())
```

Tail sampling

Application exports all spans to a collector.

The collector waits until the trace is complete, then decides whether to keep it (tail decision).

Rules can use:

- status codes
- total duration
- attributes like endpoint, tenant, feature flag, etc.

Pros:

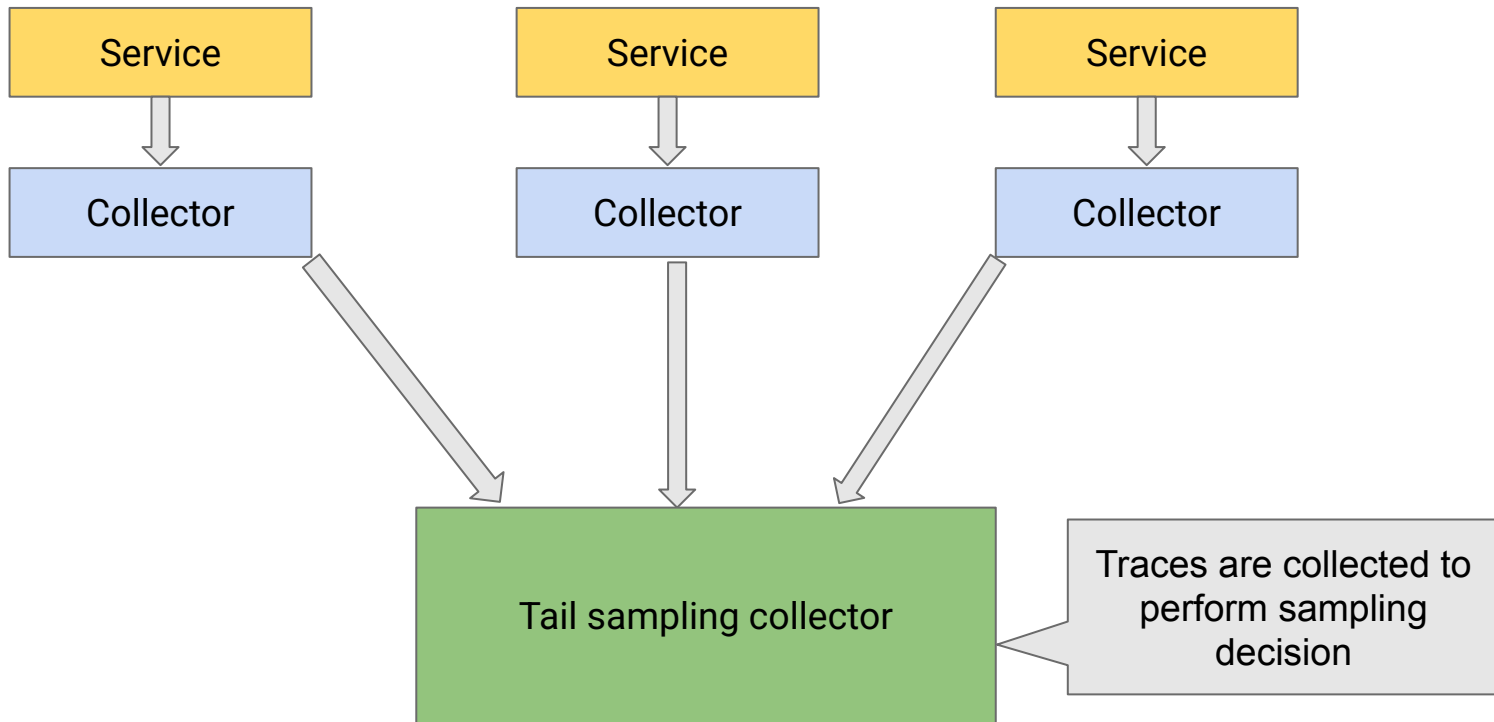
Keeps more “interesting” traces (errors, slow).
Better control of cost vs. value.

Cons:

Needs central infra (collector / pipeline).

Higher network & CPU overhead before sampling happens.

Tail sampling



~~Full~~ Dynamic sampling

Keep 100% of traces:

With errors

Above latency threshold

Traces for VIP tenants or
critical endpoints

Keep N% of other traces

Drop health checks

Sample 1% of low value
traces

Convert other traces to metrics

To manage quality of data
vs cost of storage



Domain modeling

Semantic conventions

What?

A standardized set of attribute names for common operations (e.g., HTTP requests, database queries, messaging systems).

Why?

Instead of one team using `http.request.method` and another using `request_method`, semantic conventions ensure uniformity.

Value

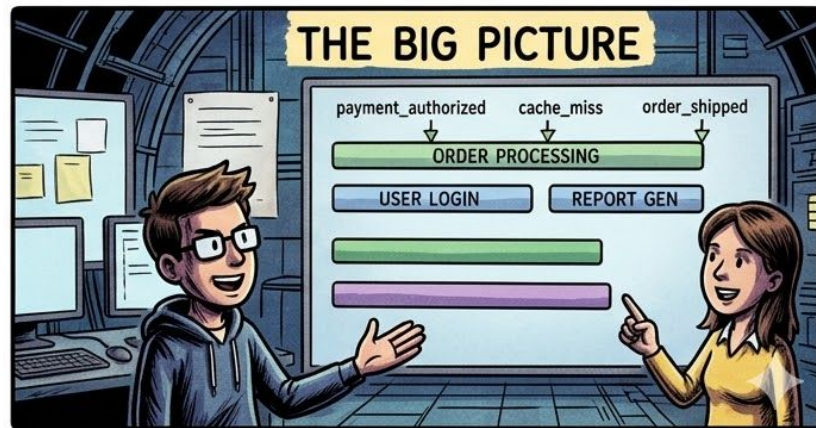
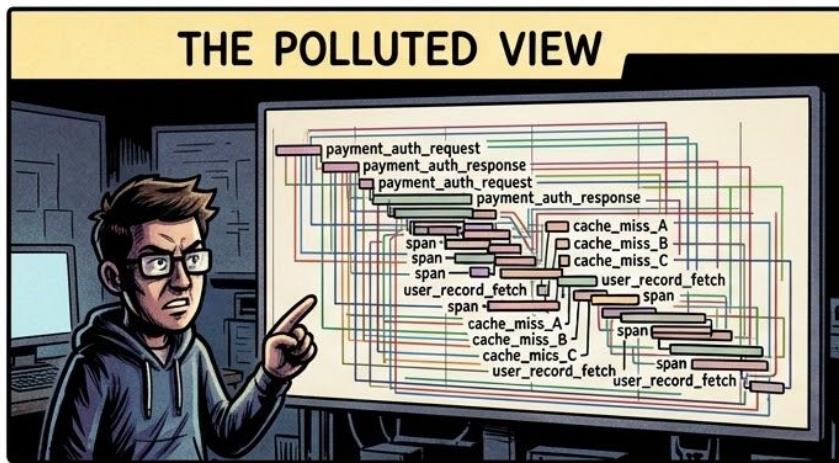
Adhering to these standards allows observability backends to automatically build out-of-the-box dashboards, service maps, and latency analyses without custom configuration.

Domain events/ span events

Not everything needs to be a separate span.

Use span events to record meaningful, point-in-time business milestones within a single operation (e.g., "payment_authorized", "cache_miss").

It helps to avoid pollution of the bigger picture.



Naming

Span names should be **low-cardinality**.

Name spans based on the class of the operation, not the specific instance.

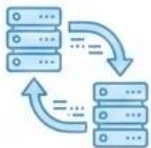
Bad: GET /user/12345

Good: GET /user/{user_id}

Better: GET_USER_DETAILS

Span boundaries

Network/Process boundaries:



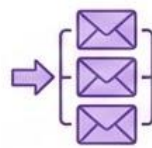
Always create spans at network boundaries (e.g., inbound HTTP requests, outbound database calls, gRPC calls).

Major Domain Logic:



Create internal spans for major algorithmic steps or business logic chunks (e.g., `calculate_fraud_score`).

Asynchronous Hops:



Create boundaries where work is handed off to queues (Producer/Consumer spans).



Avoid instrumenting every single internal function or method. "Chatty" internal spans create massive data overhead and make traces difficult to read without adding operational value

Span boundaries

Short operations

If an operation takes a few milliseconds and doesn't cross a network boundary, ask yourself if it needs to be a span.

Often, an **attribute** on the parent span or a **span event** is much more efficient.

Span boundaries

Long operations

Long-running background tasks, batch jobs, or multi-hour workflows.

Problem:

Keeping a single span open for hours can cause memory leaks in applications or the OTel Collector and delay the export of telemetry.

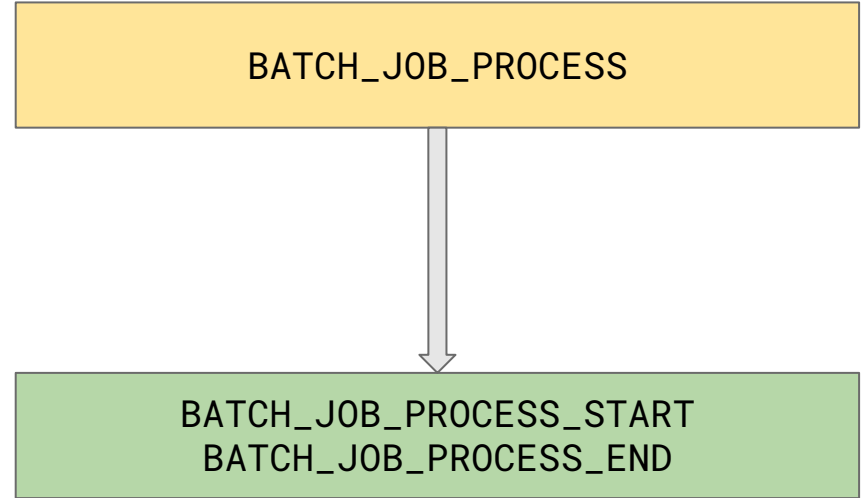
Solution

Break massive batch jobs into smaller, distinct traces (e.g., one trace per item processed).

Use **span links** to connect these smaller traces back to the primary trigger event so you don't lose the overarching relationship

Span boundaries

Long operations



TRACES AS ANALYTICS & BI SOURCE

THE PROBLEM

Latency? Errors?
But WHO? Which tenant tier?
What are they trying to do?

Need deeper
business
context!

THE SOLUTION: ENRICHED ROOT SPANS

ROOT SPAN

HTTP REQUEST
BUSINESS ATTRIBUTES:
user_group: gold
journey: checkout
tenant: acme_inc
asset_type: media_player

Add semantic
domain attributes!

FACT TABLE VIEW & EXPORT

Root Span as
"Request Row"

BUSINESS ATTRIBUTES:

request_id	user_group	latency	...
...	...	...	...

Export to Data
Warehouse!

HIGHLY QUERYABLE BUSINESS ARTIFACT

Which tenant tier
has the most
checkout latency?

USER JOURNEY FUNNELS

Map exact path!
Map exact path!
HOME PAGE
PRODUCT
Checkout Path
Latent Path (Errors!)

A/B TESTING & COHORTS

PROVE FEATURE
IMPACT WITH
COHORTS!

BETA UI
95% CONVERSION

CONTROL UI
80% CONVERSION

More conversion!

Analyze
performance with
business context!

Instrumentation strategies

Auto instrumentation

Before your code runs,
it imports auto-instrumentation packages
(HTTP clients, web frameworks, DB drivers,
etc.)
which monkey-patch their functions
at runtime without changing your code

```
opentelemetry-instrument  
--traces_exporter console  
--metrics_exporter none  
--logs_exporter none python app.py
```

Auto instrumentation

The Java agent adds
telemetry
for many popular libraries
and frameworks without
code changes
by weaving their
bytecode at startup.

```
java  
-javaagent:path/to/opentelemetry-javaagent.jar  
-Dotel.exporter.otlp.endpoint=http://otel-collector:4317  
-Dotel.service.name=your-service-name -jar app.jar
```

Auto instrumentation limits

No business context

Only sees library calls, so you don't get spans/metrics for your core domain logic.

Limited control

You can't properly design span names, hierarchies, or attributes.

Coverage gaps & fragility

It only works for supported libraries and "typical" usage patterns

Manual: log correlation

```
from opentelemetry.instrumentation.logging import LoggingInstrumentor
```

```
# This call patches the logging module to inject  
# otelTraceID, otelSpanID, and otelServiceName  
# into every log record automatically
```

```
LoggingInstrumentor().instrument(set_logging_format=True) 1
```

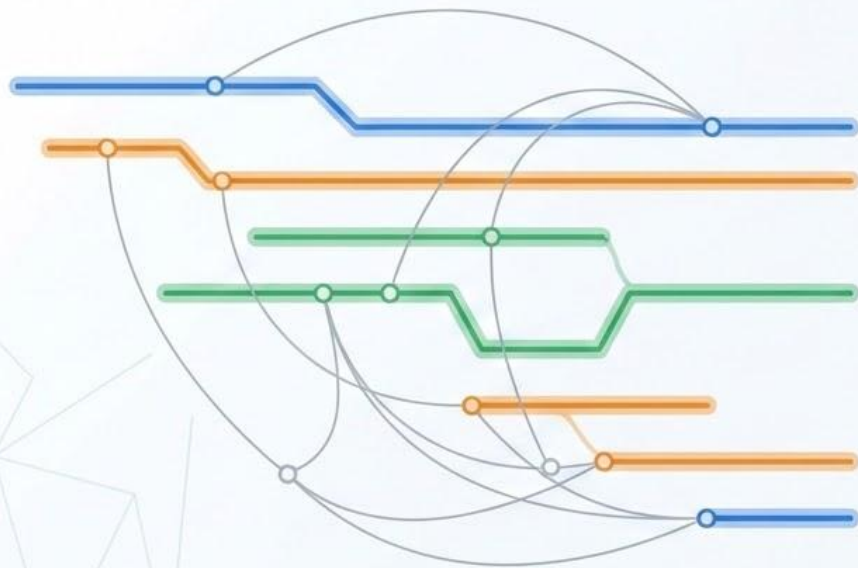
```
tracer = trace.get_tracer(__name__)
```

```
with tracer.start_as_current_span("process-order"): 2
```

```
    logger.info("Processing order #12345")
```


Advanced instrumentation strategies

When span links come into play



Non-hierarchical: Links do not establish a parent-child relationship



Multi-directional: A span can link to multiple other spans



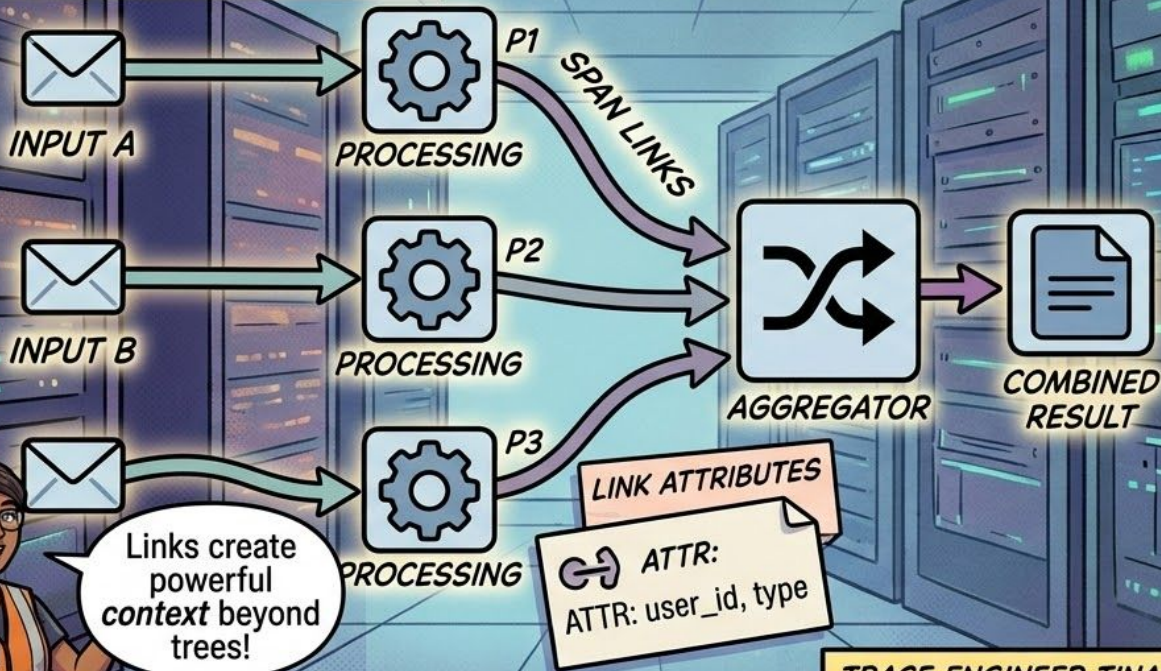
Cross-trace: Links can connect spans from different traces



Attributable: Each link can carry its own attributes

SPAN LINKS: BEYOND THE TREE

SCATTER-GATHER WORKFLOW (AGGREGATION)

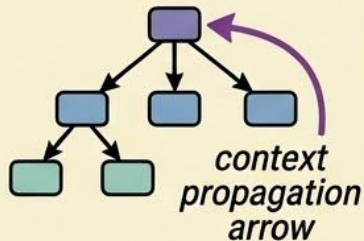


Links create powerful context beyond trees!

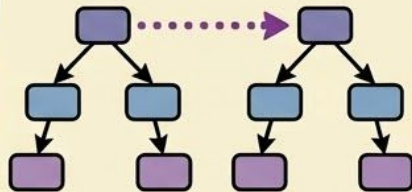
TRACE ENGINEER TINA

TRACE TYPES

PARENTS:



LINKS:



Even when parent-child fails, this *preserves* flow!

SENIOR DEV DAVE

Span links instrumentation

```
def process_batch(messages):  
    links = [  
        Link(  
            context=msg['span_context'],  
            attributes={  
                'messaging.message_id': msg['id'],  
                'messaging.source': msg['source'],  
            }  
        )  
        for msg in messages  
    ]  
  
    with tracer.start_as_current_span(  
        'process-batch',  
        kind=SpanKind.CONSUMER,  
        links=links,  
    ) as span: 2  
        span.set_attribute('batch.size', len(messages))  
  
        for message in messages:  
            process_message(message)
```

Cross trace correlations

Trace ID

Define a strictly linked, technical execution path (a single request life-cycle).

However, modern business workflows often break this technical continuity (e.g., a user orders an item on Monday, and the warehouse system ships it on Wednesday).

Cross trace correlations

Business correlation ID

For asynchronous, multi-day workflows, rely on business correlation IDs (e.g., `order_id`, `customer_id`, `workflow_id`).

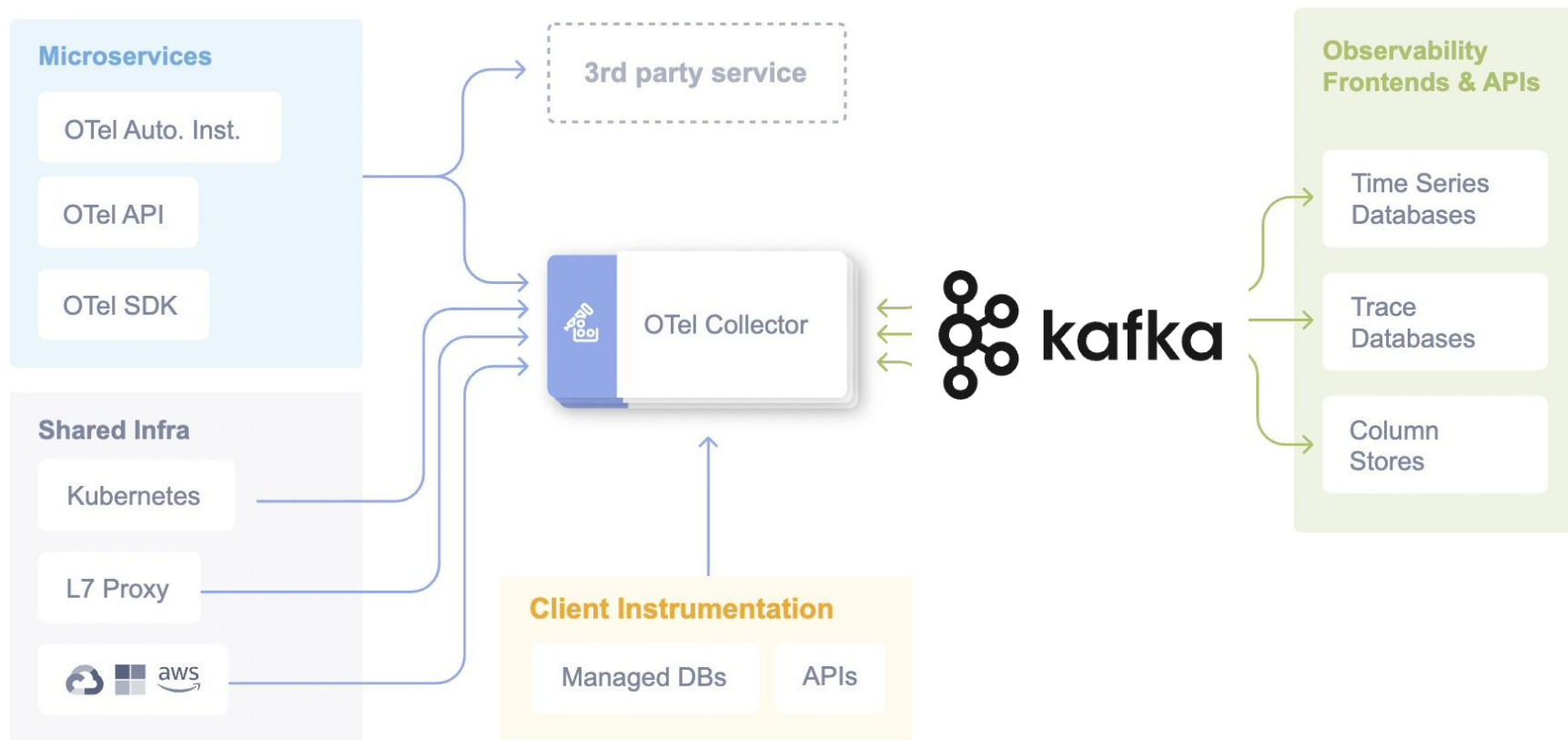
Attributes

When troubleshooting a complex, long-running business failure, you can query your observability backend for `order_id="xyz"` to retrieve all the distinct, disconnected Trace IDs that played a role in that entity's lifecycle.

```
span.set_attribute("my-business-correlation-id", my_id)
```

Preprocessing/enrichment

Enrichment



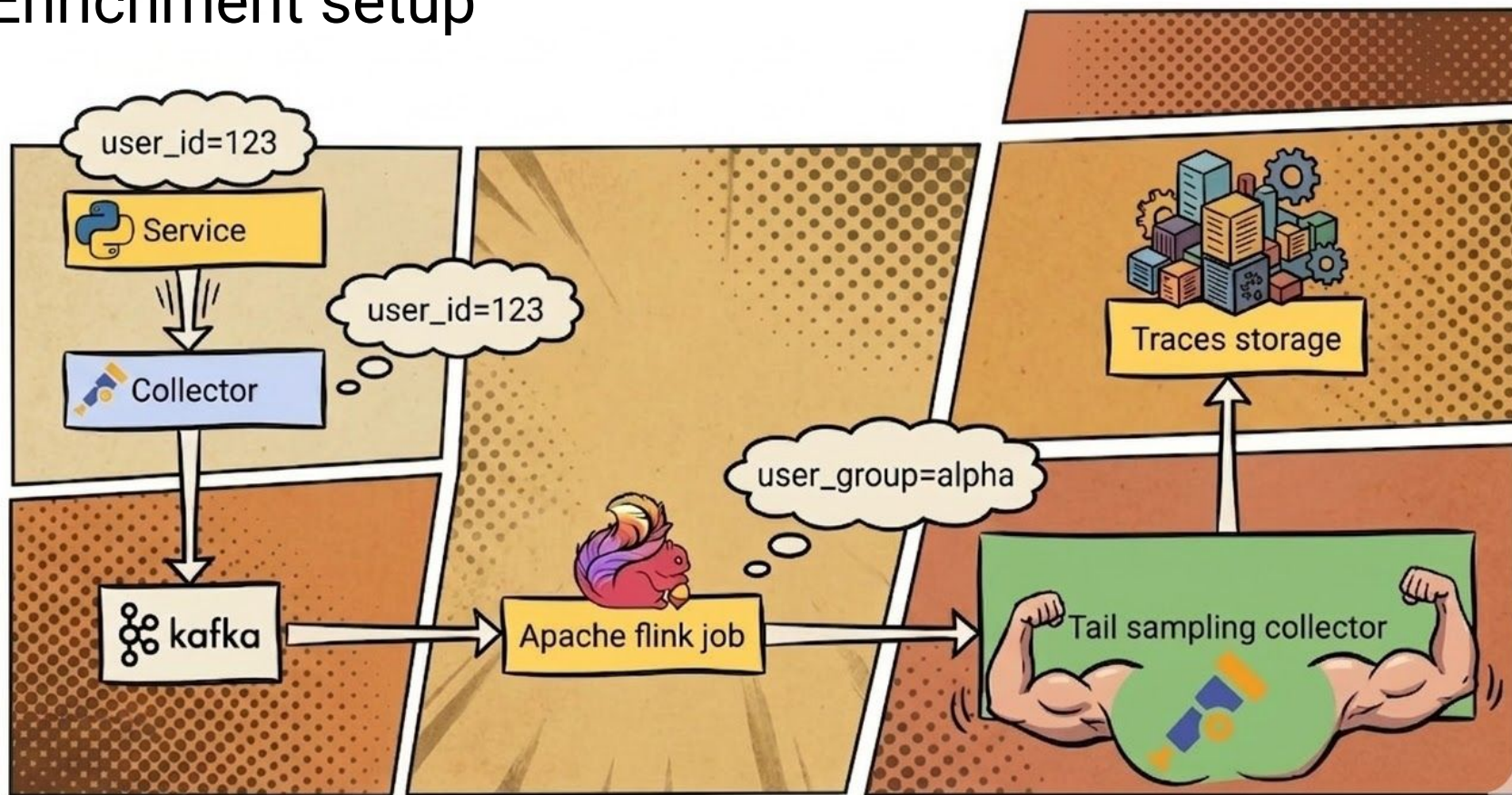
Enrichment

Enrichment pipeline Raw OTLP → Kafka → Flink/stream job → enriched OTLP → storage backend.

Use cases External joins: adding tenant tier, data-classification.

Guardrails Never mutate trace_id/span_id; only add attributes; centralize PII redaction here.

Enrichment setup



Enrichment setup: service

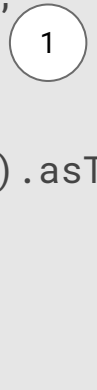
```
@app.get("/hello")
async def hello(user_id: int = Query(..., description="User ID")):
    with tracer.start_as_current_span("hello-endpoint") as span:
        span.set_attribute("user_id", user_id)

    return JSONResponse(
        {
            "message": "Hello from FastAPI with OpenTelemetry",
            "user_id": user_id,
        }
    )
```

Enrichment setup: apache flink job

```
String userIdValue = null;
ArrayNode newAttrs = MAPPER.createArrayNode();

for (JsonNode attr : attrs) {
    String key = attr.path("key").asText();
    if ("user_id".equals(key)) {
        JsonNode v = attr.path("value");
        if (v.has("intValue")) {
            userIdValue = v.get("intValue").asText();
        }
        // drop user_id
    } else {
        newAttrs.add(attr);
    }
}
```



Enrichment setup: apache flink job

```
if (userIdValue != null) {  
    String group;  
    try {  
        long id = Long.parseLong(userIdValue);  
        group = (id % 2 == 0) ? "beta" : "alpha";  
    } catch (NumberFormatException e) {  
        group = "unknown";  
    }  
  
    ObjectNode userGroupAttr = MAPPER.createObjectNode();  
    userGroupAttr.put("key", "user_group");  
    ObjectNode val = MAPPER.createObjectNode();  
    val.put("stringValue", group);  
    userGroupAttr.set("value", val);  
    newAttrs.add(userGroupAttr);  
    ...  
}
```

4

5

6

Instrumentation resiliency

Graceful shutdown

```
provider = TracerProvider()
processor = BatchSpanProcessor(ConsoleSpanExporter())
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

tracer = trace.get_tracer(__name__)

def handle_shutdown(sigum, frame): 2
    provider.shutdown()

signal.signal(signal.SIGTERM, handle_shutdown) 1
signal.signal(signal.SIGINT, handle_shutdown)
```

Trace quality

Trace quality

1. **Is the root span clearly defined?** Does the inbound HTTP request or queue trigger initiate a trace with no parent ID?
2. **Are span names low-cardinality?** Have all dynamic IDs and parameters been moved out of the span name and into span attributes?
3. **Are network and domain seams clearly bounded?** Are all inbound network requests tagged as `SERVER` and outbound calls to databases/APIs tagged as `CLIENT`?
4. **Are asynchronous handoffs modeled correctly?** Are messages handed off to a queue properly tagged with `PRODUCER` and `CONSUMER` kinds to measure time sitting in the queue versus time processing?
5. **Are errors accurately bubbling up?** When an exception occurs, is the span explicitly setting `status_code=ERROR` and recording the exception message so tail samplers know to keep it?
6. **Are business correlation IDs attached?** For multi-day workflows, are `order_id` or `tenant_id` attributes injected via Baggage or span attributes so the trace can be queried analytically later?

Questions?



<https://www.linkedin.com/in/makeitgit/>